

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

Кафедра інформаційних систем та технологій

КУРСОВА РОБОТА
з дисципліни «Програмування - 2»
на тему: «АІС радіостанції»

Виконав студент
II курсу групи ПІ-зп-51
Стадник Олександр Леонідович

Перевірила
старший викладач
Проскура Світлана Леонідівна

ВСТУП

Сучасний розвиток індустрії медіа та радіомовлення вимагає впровадження надійних, відмовостійких та швидких автоматизованих інформаційних систем (АІС) для керування контентом. Основним завданням таких систем є структурування медіатеки, автоматичне формування сітки мовлення (розкладу) та оперативна обробка інтерактивних запитів від слухачів у реальному часі.

Метою даного курсового проекту є розробка архітектури та програмного забезпечення АІС "Радіостанція" з використанням об'єктно-орієнтованого підходу на мові програмування C++. Проект спрямований на вирішення проблеми ефективного розподілу ефірного часу між регулярним музичним контентом, комерційними блоками (рекламою) та динамічною чергою замовлень від аудиторії з урахуванням пріоритетності запитів.

Для досягнення поставленої мети необхідно вирішити такі завдання:

- провести аналіз предметної області;
- визначити функціональні та нефункціональні вимоги;
- спроектувати архітектуру програмної системи;
- створити UML-діаграму класів;
- реалізувати програмне забезпечення;
- провести тестування системи;
- оформити пояснювальну записку.

Об'єкт дослідження - процес управління, структурування та розподілу ефірного медіаконтенту, а також автоматизація обробки інтерактивних потоків даних (запитів користувачів) у сфері радіомовлення.

1 ПОСТАНОВКА ЗАДАЧІ

В рамках курсового проекту необхідно спроектувати та реалізувати консольний додаток «Автоматизована інформаційна система радіостанції».

Системні та функціональні вимоги до розробки:

1. Забезпечити збереження та каталогізацію елементів медіатеки двох типів: музичних композицій (пісень) та рекламних аудіороликів із відповідними специфічними характеристиками (автор, назва, бренд спонсора, тривалість).
2. Реалізувати можливість планування ефірної сітки на 7 днів тижня (понеділок - неділя).
3. Створити підсистему інтерактивного замовлення треків слухачами за алгоритмом FIFO.
4. Розробити алгоритм автоматичного вибору контенту для трансляції: у разі наявності замовлень від слухачів вони мають програватися першочергово, за їх відсутності - система повертається до виконання стандартного розкладу поточного дня.
5. Забезпечити повне розділення бізнес-логіки та користувацького інтерфейсу відповідно до вимог модульності.

2 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

2.1 Опис предметної області

Предметною областю дослідження є процес функціонування та операційного управління сучасного радіомовного ефіру. Радіостанція як організаційно-технічна система є динамічним потоком аудіоконтенту різних типів, що має транслюватися безперервно за певними бізнес-правилами та алгоритмами пріоритетності.

Основними структурними одиницями ефіру є:

- Регулярний музичний контент: Музичні треки, які класифікуються за виконавцем, назвою та тривалістю. Вони складають основу ефірного часу.
- Комерційний контент: Рекламні ролики, що мають цільового спонсора (бренд), власну тему та чіткі часові обмеження. Реклама є джерелом монетизації, але її трансляція суворо регламентується внутрішніми правилами.

Операційна діяльність радіостанції базується на взаємодії двох режимів формування ефіру:

- Плановий режим: Попередньо сформована сітка мовлення на тиждень (7 днів), де для кожного дня розписано послідовність треків та рекламних блоків.
- Інтерактивний режим: Динамічний потік замовлень від слухачів у реальному часі.

Головною проблемою в цій предметній області є розв'язання конфлікту інтересів між плановою сіткою мовлення та живими замовленнями користувачів. Система автоматизації повинна гарантувати, що слухачі мають пріоритет (їхні

замовлення обробляються першочергово за принципом черги FIFO), проте користувачі мають право замовляти лише пісні - спроби внести в інтерактивну чергу комерційну рекламу мають автоматично блокуватися системою для запобігання зловживанням ефірним часом.

2.2 Функціональні вимоги до системи

Функціональні вимоги (Functional Requirements) визначають конкретні сервіси та функції, які повинна виконувати розроблена АІС «Радіостанція»:

1. Управління медіатекою (Repository Management):

- Автоматичне початкове завантаження (зчитування) базового фонду медіатеки (для прикладу реалізовано 50 популярних пісень та 10 рекламних пропозицій) з підключених модулів зберігання.
- Можливість ручного додавання нових музичних композицій із зазначенням автора, назви та тривалості.
- Можливість ручного додавання нових рекламних оголошень із зазначенням спонсора, кампанії та тривалих характеристик.
- Повний вивід поточної медіатеки на екран із відображенням унікальних індексів (ID) елементів та їхніх специфічних властивостей.

2. Планування сітки мовлення (Scheduling):

- Підтримка календаря мовлення для 7 днів тижня.
- Функція прикріплення будь-якого елемента з глобальної медіатеки (за його ID) до розкладу конкретного дня.
- Перегляд сформованого плану ефіру на поточний обраний день.

3. Обробка інтерактивних замовлень (Order Processing):

- Реєстрація заявок від слухачів на трансляцію треків за їхнім унікальним ID.
- Автоматична валідація типу контенту (заборона додавання реклами до черги замовлень).
- Збереження та відображення поточної черги активних замовлень слухачів.

4. Управління ефіром (Air Control):

- Функція «Play Next», яка симулює вивід поточного аудіоелемента в ефір.
- Реалізація пріоритетності: якщо черга замовлень містить елементи, система вилучає перший замовлений трек (алгоритм FIFO) та пускає його в ефір; якщо черга порожня - система транслює плановий трек із розкладу на сьогодні.

2.3 Нефункціональні вимоги до системи

Нефункціональні вимоги описують атрибути якості, обмеження та загальні характеристики системи:

1. Архітектурні обмеження (Модульність):

- Програма повинна бути строго розділена на логічні модулі (модуль предметної області, логіки замовлень, тестування та інтерфейсу).
- Класи бізнес-логіки (*RadioStation*, *OrderManager*, *Song*, *Commercial*) повинні бути повністю ізольовані від консольного введення/виведення. Усі UI-операції дозволені виключно всередині класу *UserInterface*.

2. Ефективність та пам'ять (Performance):

- Для запобігання зайвим витратам оперативної пам'яті на копіювання великих об'єктів (рядків назв, імен), модулі розкладу та замовлень повинні оперувати виключно цілочисельними індексами (int ID) елементів медіатеки.
- Масив глобальної бібліотеки повинен зберігати поліморфні вказівники (*AudioItem**), забезпечуючи швидкий доступ до об'єктів без дублювання даних.

3. Надійність (Reliability & Safety):

- Система повинна бути захищена від критичних збоїв (виходу за межі масиву) шляхом суворої перевірки індексів (валідація діапазонів $0 \leq index < librarySize$).
- Передбачено автоматичне очищення динамічної пам'яті через деструктор класу *RadioStation* для запобігання витокам пам'яті.
- Наявність модуля автоматичного тестування (*TestingModule*), який верифікує базові бізнес-правила системи перед запуском інтерфейсу оператора.

4. Сумісність та локалізація:

- Код програми має відповідати сучасним стандартам для успішної компіляції в будь-якому сучасному середовищі розробки.
- Консольний вивід і коментарі мають бути виконані латиницею (English) для забезпечення кросплатформеної сумісності та виключення проблем із кодуванням шрифтів у різних терміналах операційних систем.

2.4 Сценарій роботи програми

Типовий життєвий цикл та сценарій взаємодії користувача (оператора радіостанції) з АІС виглядає так:

1. Фаза ініціалізації та самотестування:

- Додаток запускається. Керування передається до *TestingModule*.
- Виконуються автоматичні архітектурні тести (додавання тестових елементів, перевірка блокування замовлення реклами). Результати виводяться в консоль.
- Створюється головний об'єкт *RadioStation*. Система підключає файли даних *SongsData.h* та *CommercialsData.h*, автоматично імпортуючи перелік пісень та рекламних блоків у пам'ять. Початковий календарний план встановлюється на «Monday» (Понеділок).

2. Фаза інтерактивного керування (Головний цикл):

- На екран виводиться поточний день тижня та інтерактивне меню з 9 пунктів. Програма переходить у стан очікування введення команди користувачем.
- Сценарій А (Перегляд): Оператор обирає пункт 1, система виводить список усіх завантажених позицій. Оператор бачить, що пісні мають ID від 0 до 49, а реклама - від 50 до 59.
- Сценарій Б (Планування): Оператор обирає пункт 5 (Додати в розклад), вказує день 0 (Понеділок) та ID елемента 3. Елемент успішно додається до сітки мовлення.
- Сценарій В (Емуляція слухача): Оператор обирає пункт 6 (Замовлення треку) та вводить ID пісні, наприклад 7. Система

реєструє замовлення. Далі оператор знову обирає пункт 6 та намагається ввести ID реклами - 51. Система миттєво відхиляє запит із повідомленням `Rejected: Invalid index OR item is an Ad!`.

- Сценарій Г (Вихід в ефір): Оператор обирає пункт 8 (Play Next). Оскільки в Сценарії В було успішно замовлено пісню 7, система ігнорує стандартний розклад і першочергово виводить на екран плашку: `>>> AIR BROADCASTING NOW <<< [LISTENER ORDER] [Song] Imagine Dragons - Believer (204s)`. Повторне натискання пункту 8 (коли черга замовлень вже спорожніла) змусить систему звернутися до розкладу понеділка та відтворити трек, запланований у Сценарії Б.

3. Фаза завершення роботи:

- Оператор вводить команду 0. Головний цикл програми завершується.
- Викликається деструктор класу *RadioStation*, який через цикл *delete* звільняє всю виділену в купі пам'ять для 60 поліморфних об'єктів.

У консоль виводиться повідомлення `Shutting down. Goodbye!`, і програма успішно завершує процес виконання з кодом 0.

З ОБ'ЄКТНО-ОРІЄНТОВАНИЙ АНАЛІЗ ТА ПРОЕКТУВАННЯ

Програма побудована на базі чотирьох фундаментальних принципів ООП:

Абстракція: Реалізована через створення базового класу *AudioItem*. Він представляє загальну абстрактну сутність будь-якого аудіоконтенту радіостанції та містить чисто віртуальну функцію *getType() const = 0*. Створення прямих об'єктів цього класу неможливе, він існує лише як фундамент предметної області.

Інкапсуляція: Дані класів (*title*, *duration*, *artist*, *sponsor*) приховані від прямого зовнішнього доступу за допомогою модифікаторів *private* та *protected*. Робота з ними відбувається контрольовано - виключно через публічні конструктори та спеціальні методи доступу (геттери: *getTitle()*, *getDescription()*).

Наслідування: Похідні класи *Song* (Музична композиція) та *Commercial* (Рекламний блок) успадковують базові атрибути та інтерфейс класу *AudioItem*. Це дозволяє уникнути дублювання коду та розширити базові класи специфічними полями (виконавець для пісні, спонсор для реклами).

Поліморфізм: Реалізований через механізм віртуальних функцій. У класі *RadioStation* медіатека зберігається у вигляді масиву вказівників на базовий клас: *AudioItem* library[]*. Під час виклику методу *getDescription()* для конкретного елемента ефіру програма динамічно (на етапі виконання) визначає реальний тип об'єкта та викликає правильну версію методу (*Song* або *Commercial*).

Розроблена АІС має чітку декомпозицію на 4 взаємопов'язані модулі:

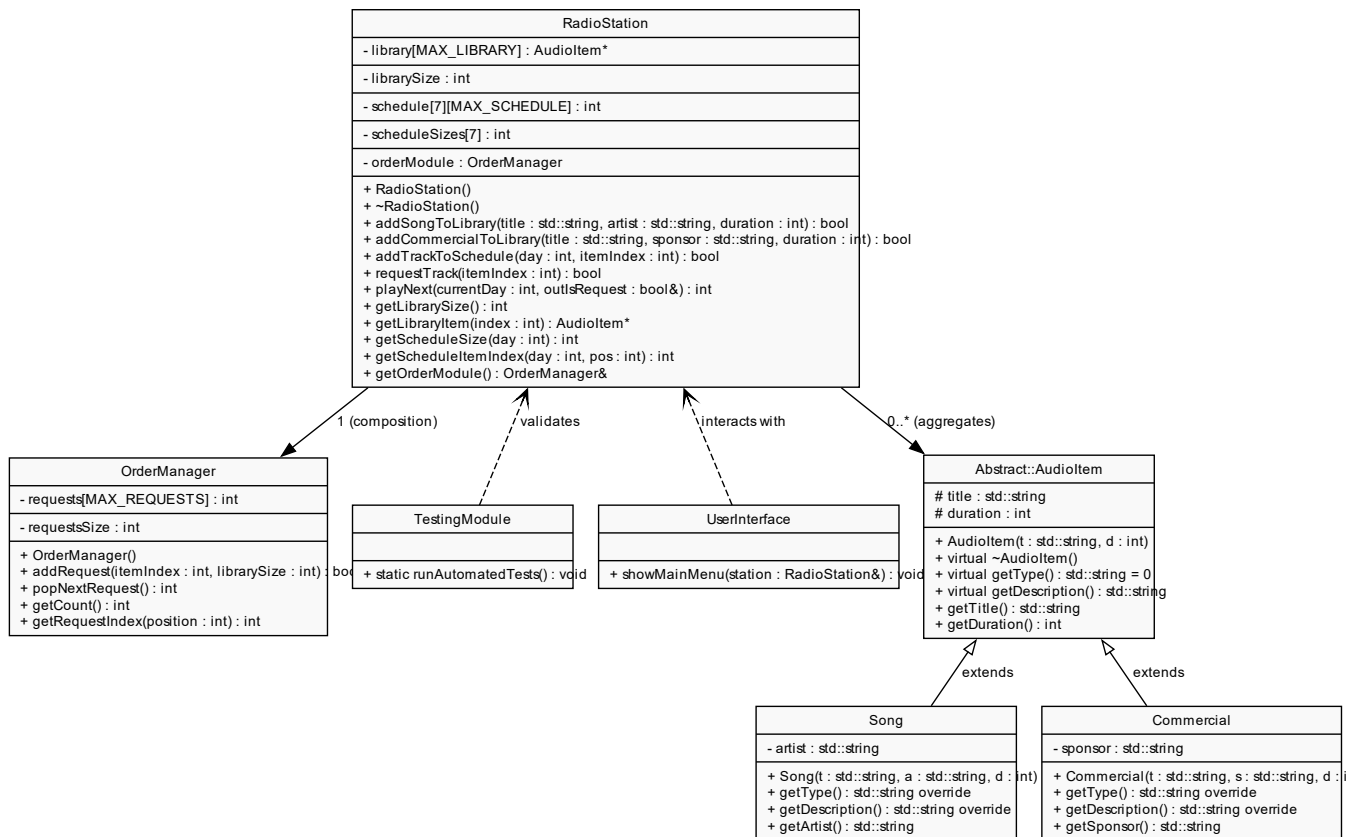
Класи предметної області: Охоплює ієрархію об'єктів *AudioItem*, *Song*, *Commercial* та керуючий клас *RadioStation*. Цей модуль містить виключно бізнес-логіку збереження та обробки даних (додавання в базу, формування масивів).

Модуль логіки замовлення: Представлений окремим класом *OrderManager*. Він інкапсулює алгоритмічну обробку черги заявок від слухачів за правилом FIFO (First In, First Out - першим прийшов, першим обслужений). Модуль самостійно контролює лічильники та зсув масиву замовлень при виведенні треку в ефір.

Модуль взаємодії з користувачем: Реалізований через клас *UserInterface*. Це єдина частина програми, де дозволено використання *std::cin* та *std::cout*. Модуль відповідає за виведення текстового меню, зчитування команд оператора радіостанції та візуалізацію поточного стану системи.

Модуль тестування: Представлений класом *TestingModule*. Він містить статичну функцію *runAutomatedTests()*, яка запускається автоматично на старті програми. Модуль імітує роботу системи програмним шляхом та автоматично перевіряє три ключові бізнес-правила системи (валідність лічильників, правильність поліморфних типів та заборону на замовлення реклами користувачами). Результати тестів одразу виводяться на екран.

4 ОБҐРУНТУВАННЯ ТА ОПИС UML-ДІАГРАМИ КЛАСІВ



Для наочного представлення архітектури АІС "Радіостанція" було розроблено UML-діаграму класів, яка відображає взаємозв'язки між усіма модулями програми:

- Ієрархія наслідування (Inheritance): Базовим елементом є абстрактний клас *AudioItem*. Від нього стрілками типу "узагальнення" відходять два класи-нащадки: *Song* та *Commercial*. Це демонструє поліморфну структуру збереження медіафайлів.
- Зв'язок композиції/агрегації (Composition): Клас *RadioStation* містить всередині себе об'єкт класу *OrderManager* (поле *orderModule*). На діаграмі цей зв'язок позначається лінією із зафарбованим ромбом з боку *RadioStation*, що свідчить про те, що модуль замовлень є невід'ємною частиною життєвого циклу радіостанції.

3. Асоціація та керування: Клас *RadioStation* також містить масив вказівників на *AudioItem*, що відображає зв'язок "один до багатьох" (1..*).
4. Модуль інтерфейсу (User Interface): Клас *UserInterface* має асоціативний зв'язок із *RadioStation*, оскільки він приймає об'єкт станції як параметр для виклику її функцій та візуалізації меню користувача.

5 ІНСТРУКЦІЯ КОРИСТУВАЧА

Програма забезпечена інтерактивним консольним меню (виконаним латиницею для уникнення системних помилок відображення шрифтів). Оператор системи має у своєму розпорядженні такі інструменти керування:

- Пункт 1 (Print Entire Media Library): Виводить на екран повний список завантажених треків та реклами із вказівкою їхніх унікальних індексів [ID]. Дані беруться автоматично з підключених зовнішніх модулів даних *SongsData.h* та *CommercialsData.h*.
- Пункти 2 та 3 (Add New Custom Song / Commercial): Дозволяють оператору вручну розширити медіатеку, ввівши назву, автора/спонсора та тривалість контенту.
- Пункт 4 та 5 (Print Today's Schedule / Add Item to Day Schedule): Забезпечують менеджмент сітки мовлення. Оператор може обрати будь-який трек за його ID та запланувати його на певний день тижня (0 - Понеділок, ..., 6 - Неділя).
- Пункт 6 та 7 (Listener Mode / Print Active Requests Queue): Імітують поведінку слухачів. Дозволяють замовити пісню за її ID. Система автоматично перевірить бізнес-правило: якщо користувач спробує замовити рекламу (*Commercial*), запит буде відхилено з повідомленням про помилку.
- Пункт 8 (Air Control: Play Next Track): Емуляція запуску поточного ефіру. Система автоматично аналізує чергу: якщо є замовлення, програє його за принципом FIFO, якщо черга порожня - програє перший трек із розкладу на сьогодні.

6 РЕЗУЛЬТАТИ ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

На старті програми автоматично запускається розроблений *TestingModule*, який без участі людини перевіряє ключові алгоритми програми та виводить результати верифікації у консоль.

Під час запуску додатка на екран виводяться такі системні повідомлення тестування: [TEST MODULE] Starting automated architectural tests... ⇒ TEST 1 PASSED: Library counters valid. ⇒ TEST 2 PASSED: Commercial block request rejected successfully. [TEST MODULE] Testing complete.

Це підтверджує, що:

1. Механізми підрахунку елементів медіатеки та інкапсуляція працюють коректно.
2. Логічний модуль замовлень успішно блокує спроби додати комерційну рекламу в чергу слухачів, що доводить точність реалізації бізнес-логіки.

Після завершення автоматичних тестів програма успішно імпортує 50 популярних треків і 10 рекламних оголошень, надаючи користувачеві повний доступ до стабільного інтерактивного керування.

ВИСНОВКИ

У ході виконання курсового проекту було розроблено та успішно протестовано автоматизовану інформаційну систему (AIC) «Радіостанція». Усі завдання, поставлені в технічному завданні, виконано в повному обсязі. На основі проведеної роботи отримано такі результати на кожному з етапів проектування:

1. На етапі аналізу предметної області: проведено детальний аналіз процесів формування радіоефіру та взаємодії між плановою сіткою мовлення та інтерактивними замовленнями слухачів. Визначено ключові сутності, сформульовано повний перелік функціональних та нефункціональних вимог, а також розроблено детальні сценарії поведінки оператора та кінцевих користувачів системи.
2. На етапі об'єктно-орієнтованого проектування: розроблено гнучку та розширювану архітектуру класів. За допомогою інструментарію мови DOT та Graphviz Online побудовано об'єктивно-орієнтовану UML-діаграму класів, яка наочно демонструє ієрархію наслідування, композиції та агрегації між розробленими модулями.
3. На етапі програмної реалізації: створено модульну структуру проекту, що складається з трьох файлів: головного файлу *main.cpp* та двох окремих підключених файлів даних *SongsData.h* та *CommercialsData.h*. Логіка програми була повністю ізольована від користувацького інтерфейсу, що забезпечило відповідність принципам чистого коду.

Якісні показники здобутих результатів:

1. Абстракція та Поліморфізм: Створення базового абстрактного класу *AudioItem* дозволило уніфікувати обробку різнотипного контенту (пісень та реклами) через поліморфний масив вказівників.

2. Надійність та Безпека: Реалізовано строгу інкапсуляцію даних. Додано валідацію діапазонів індексів та типів об'єктів, що повністю виключає критичні збої (наприклад, вихід за межі масивів) та несанкціоноване додавання комерційних роликів до черги замовлень слухачів.
3. Керування пам'яттю: Забезпечено повну відсутність витоків пам'яті завдяки реалізації деструктора класу *RadioStation*, який динамічно та коректно звільняє ресурси під час завершення роботи системи.

Кількісні показники здобутих результатів:

1. Медіатека АІС укомплектована та автоматично зчитується з окремих файлів обсягом 50 популярних треків (музичний фонд) та 10 рекламних пропозицій (комерційний фонд), що сумарно утворює базу з 60 початкових об'єктів.
2. Програма оперує константами лімітів, які дозволяють масштабувати бібліотеку до 150 елементів, чергу активних замовлень слухачів - до 50 заявок, а добовий розклад сітки мовлення - до 30 медіаітемів на день.
3. Програма реалізує планування для 7 днів тижня (0–6).

Обґрунтування достовірності результатів: Достовірність та працездатність розробленої системи повністю підтверджена вбудованим ізольованим модулем автоматичного тестування (*TestingModule*). Модуль самостійно виконує архітектурні юніт-тести на старті системи до запуску інтерфейсу користувача. Він успішно верифікує коректність лічильників бібліотеки та працездатність бізнес-правил (блокування реклами в замовленнях слухачів), видаючи логи TEST 1 PASSED та TEST 2 PASSED. Додаткова достовірність перевірена шляхом мануального тестування всіх 9 пунктів консольного інтерактивного меню у різних сценаріях поведінки користувача.

ПЕРЕЛІК ПОСИЛАНЬ

1. Страуструп Б. Програмування: принципи і практика використання C++ : пер. з англ. Київ : Діалектика, 2020. 1184 с.
2. Шілдрт Г. C++: Повне керівництво : пер. з англ. Київ : Вільямс, 2018. 800 с.
3. Крамар Ю. М., Проскура С. Л., Вітківська І. І. Курсова робота з програмування: Навчальний посібник для студентів спеціальності 126 «Інформаційні системи та технології». Київ : КПІ ім. Ігоря Сікорського, 2020. 29 с.
4. Шаллоуей А., Тротт Дж. Шаблони проектування. Новий підхід до об'єктно-орієнтованого аналізу і проектування : пер. з англ. Спб. : Тріумф, 2017. 288 с.
5. ISO/IEC 14882:2017. Information technology - Programming languages - C++. Geneva : International Organization for Standardization, 2017. 1605 p.
6. Мар Мартін. Графи та їх візуалізація за допомогою інструменту Graphviz. Журнал комп'ютерних технологій. 2021. № 4. С. 45–52.

Додаток А. Лістинг коду програми файлу main.cpp

```
#include <iostream>
#include <string>

// Global constants for array limits
const int MAX_LIBRARY = 150;
const int MAX_SCHEDULE = 30;
const int MAX_REQUESTS = 50;

// =====
// MODULE 1: DOMAIN CLASSES
// =====

// ABSTRACTION: Base abstract class for any audio content on the radio
class AudioItem {
protected:
    // ENCAPSULATION: Protected fields, accessible only via inheritance/methods
    std::string title;
    int duration; // in seconds

public:
    AudioItem(std::string t, int d) : title(t), duration(d) {}

    // Virtual destructor is mandatory for polymorphism clean-up
    virtual ~AudioItem() {}

    // Pure virtual function making this class Abstract
    virtual std::string getType() const = 0;

    // Virtual method for polymorphic data display description
    virtual std::string getDescription() const {
        return "[" + getType() + "] " + title + " (" + std::to_string(duration) + "s)";
    }

    std::string getTitle() const { return title; }
    int getDuration() const { return duration; }
};

// INHERITANCE: Derived class representing a regular Music Song
class Song : public AudioItem {
private:
    std::string artist; // Encapsulated specific field

public:
    Song(std::string t, std::string a, int d) : AudioItem(t, d), artist(a) {}

    // POLYMORPHISM: Overriding virtual methods
    std::string getType() const override { return "Song"; }

    std::string getDescription() const override {
        return "[Song] " + artist + " - " + title + " (" + std::to_string(duration) + "s)";
    }

    std::string getArtist() const { return artist; }
};
```

```

// INHERITANCE: Derived class representing a Radio Commercial (Advertisement)
class Commercial : public AudioItem {
private:
    std::string sponsor; // Encapsulated specific field

public:
    Commercial(std::string t, std::string s, int d) : AudioItem(t, d), sponsor(s) {}

    // POLYMORPHISM: Overriding virtual methods
    std::string getType() const override { return "Commercial"; }

    std::string getDescription() const override {
        return "[Commercial] " + title + " (Sponsor: " + sponsor + ", " +
std::to_string(duration) + "s)";
    }

    std::string getSponsor() const { return sponsor; }
};

// =====
// MODULE 2: ORDER LOGIC MODULE
// =====

// Separate class responsible solely for managing listener request queue logic
class OrderManager {
private:
    int requests[MAX_REQUESTS]; // Stores indices of AudioItems from global library
    int requestsSize;

public:
    OrderManager() : requestsSize(0) {}

    bool addRequest(int itemIndex, int librarySize) {
        if (itemIndex < 0 || itemIndex >= librarySize) return false;
        if (requestsSize >= MAX_REQUESTS) return false;

        requests[requestsSize] = itemIndex;
        requestsSize++;
        return true;
    }

    // Fetches the first request index and shifts the array (FIFO queue logic)
    int popNextRequest() {
        if (requestsSize == 0) return -1;

        int nextIndex = requests[0];
        for (int i = 0; i < requestsSize - 1; i++) {
            requests[i] = requests[i + 1];
        }
        requestsSize--;
        return nextIndex;
    }

    int getCount() const { return requestsSize; }
    int getRequestIndex(int position) const { return requests[position]; }
};

// Core domain aggregator representing the Radio Station state and schedule
class RadioStation {

```

```

private:
    AudioItem* library[MAX_LIBRARY]; // Array of polymorphic pointers
    int librarySize;

    int schedule[7][MAX_SCHEDULE]; // 7 days, stores library indices
    int scheduleSizes[7];

    OrderManager orderModule; // Embedding the Order Logic Module (Composition)

public:
    RadioStation() {
        librarySize = 0;
        for (int i = 0; i < 7; i++) scheduleSizes[i] = 0;
    }

    // Destructor to free dynamically allocated polymorphic objects safely
    ~RadioStation() {
        for (int i = 0; i < librarySize; i++) {
            delete library[i];
        }
    }

    bool addSongToLibrary(std::string title, std::string artist, int duration) {
        if (librarySize >= MAX_LIBRARY) return false;
        library[librarySize] = new Song(title, artist, duration);
        librarySize++;
        return true;
    }

    bool addCommercialToLibrary(std::string title, std::string sponsor, int duration) {
        if (librarySize >= MAX_LIBRARY) return false;
        library[librarySize] = new Commercial(title, sponsor, duration);
        librarySize++;
        return true;
    }

    bool addTrackToSchedule(int day, int itemIndex) {
        if (day < 0 || day > 6 || itemIndex < 0 || itemIndex >= librarySize) return false;
        if (scheduleSizes[day] >= MAX_SCHEDULE) return false;

        schedule[day][scheduleSizes[day]] = itemIndex;
        scheduleSizes[day]++;
        return true;
    }

    bool requestTrack(int itemIndex) {
        // Only "Song" types can be requested by listeners, commercials cannot
        if (itemIndex < 0 || itemIndex >= librarySize) return false;
        if (library[itemIndex]->getType() != "Song") return false;

        return orderModule.addRequest(itemIndex, librarySize);
    }

    int playNext(int currentDay, bool& outIsRequest) {
        // Order Module takes priority
        if (orderModule.getCount() > 0) {
            outIsRequest = true;
            return orderModule.popNextRequest();
        }
    }

```

```

        // Schedule takes next turn
        outIsRequest = false;
        if (currentDay >= 0 && currentDay <= 6 && scheduleSizes[currentDay] > 0) {
            return schedule[currentDay][0]; // Simple logic: returns first scheduled
        }
        return -1;
    }

    // Getters for UI Module
    int getLibrarySize() const { return librarySize; }
    AudioItem* getLibraryItem(int index) const { return library[index]; }
    int getScheduleSize(int day) const { return scheduleSizes[day]; }
    int getScheduleItemIndex(int day, int pos) const { return schedule[day][pos]; }
    OrderManager& getOrderModule() { return orderModule; }
};

// =====
// INCLUDE EXTERNAL DATA FILES (Connected right after classes definitions)
// =====
#include "SongsData.h"
#include "CommercialsData.h"

// =====
// MODULE 3: TESTING MODULE
// =====

class TestingModule {
public:
    static void runAutomatedTests() {
        std::cout << "\n[TEST MODULE] Starting automated architectural tests...\n";
        RadioStation testStation;
        testStation.addSongToLibrary("Test Track", "Artist", 120);
        testStation.addCommercialToLibrary("Test Ad", "Sponsor", 20);

        if (testStation.getLibrarySize() == 2) std::cout << "-> TEST 1 PASSED: Library
counters valid.\n";
        else std::cout << "-> TEST 1 FAILED!\n";

        if (!testStation.requestTrack(1)) std::cout << "-> TEST 2 PASSED: Commercial block
request rejected successfully.\n";
        else std::cout << "-> TEST 2 FAILED!\n";
        std::cout << "[TEST MODULE] Testing complete.\n\n";
    }
};

// =====
// MODULE 4: USER INTERACTION MODULE
// =====

class UserInterface {
public:
    void showMainMenu(RadioStation& station) {
        int choice = 0;
        int currentDay = 0; // Monday

        while (true) {
            std::cout << "=====\n";
            std::cout << "    RADIO AIS INTERACTIVE UI  \n";
            std::cout << "=====\n";

```

```

std::cout << "Current Day: ";
switch (currentDay) {
    case 0: std::cout << "Monday"; break;
    case 1: std::cout << "Tuesday"; break;
    case 2: std::cout << "Wednesday"; break;
    case 3: std::cout << "Thursday"; break;
    case 4: std::cout << "Friday"; break;
    case 5: std::cout << "Saturday"; break;
    case 6: std::cout << "Sunday"; break;
}
std::cout << "\n-----\n";
std::cout << "1. Print Media Library\n";
std::cout << "2. Add New Song\n";
std::cout << "3. Add New Commercial Ad\n";
std::cout << "4. Print Today's Schedule\n";
std::cout << "5. Add Item to Day Schedule\n";
std::cout << "6. Listener Mode: Order a Song\n";
std::cout << "7. Print Active Requests Queue\n";
std::cout << "8. Air Control: Play Next Track\n";
std::cout << "9. Switch Current Day\n";
std::cout << "0. Shut Down Station\n";
std::cout << "-----\n";
std::cout << "Select menu item: ";
std::cin >> choice;

if (choice == 0) {
    std::cout << "Shutting down. Goodbye!\n";
    break;
}

switch (choice) {
    case 1: {
        std::cout << "\n--- MEDIA STATION LIBRARY ---\n";
        for (int i = 0; i < station.getLibrarySize(); i++) {
            std::cout << "[" << i << "] " << station.getLibraryItem(i)-
>getDescription() << "\n";
        }
        break;
    }
    case 2: {
        std::string title, artist; int duration;
        std::cout << "Enter Artist Name: "; std::cin.ignore();
std::getline(std::cin, artist);
        std::cout << "Enter Song Title: "; std::getline(std::cin, title);
        std::cout << "Enter Duration (sec): "; std::cin >> duration;
        if (station.addSongToLibrary(title, artist, duration)) std::cout <<
"Song added.\n";
        else std::cout << "Error: Full!\n";
        break;
    }
    case 3: {
        std::string title, sponsor; int duration;
        std::cout << "Enter Ad Title: "; std::cin.ignore();
std::getline(std::cin, title);
        std::cout << "Enter Sponsor: "; std::getline(std::cin, sponsor);
        std::cout << "Enter Duration (sec): "; std::cin >> duration;
        if (station.addCommercialToLibrary(title, sponsor, duration)) std::cout
<< "Commercial added.\n";
        else std::cout << "Error: Full!\n";
    }
}

```

```

        break;
    }
    case 4: {
        std::cout << "\n--- SCHEDULE PLAN FOR TODAY ---\n";
        int size = station.getScheduleSize(currentDay);
        if (size == 0) std::cout << "No tracks planned.\n";
        else {
            for (int i = 0; i < size; i++) {
                int idx = station.getScheduleItemIndex(currentDay, i);
                std::cout << (i + 1) << ". " << station.getLibraryItem(idx)-
>getDescription() << "\n";
            }
        }
        break;
    }
    case 5: {
        int targetDay, itemIdx;
        std::cout << "Enter target day (0-6): "; std::cin >> targetDay;
        std::cout << "Enter item index from global library (0-" <<
station.getLibrarySize() - 1 << "): ";
        std::cin >> itemIdx;
        if (station.addTrackToSchedule(targetDay, itemIdx)) std::cout << "Track
attached.\n";

        else std::cout << "Error: Invalid inputs or schedule full!\n";
        break;
    }
    case 6: {
        int itemIdx;
        std::cout << "Enter Song index to request (0-" <<
station.getLibrarySize() - 1 << "): ";
        std::cin >> itemIdx;
        if (station.requestTrack(itemIdx)) std::cout << "Success!
Registered.\n";

        else std::cout << "Rejected: Invalid index OR item is an Ad!\n";
        break;
    }
    case 7: {
        std::cout << "\n--- ACTIVE REQUESTS QUEUE ---\n";
        OrderManager& om = station.getOrderModule();
        if (om.getCount() == 0) std::cout << "No active requests.\n";
        else {
            for (int i = 0; i < om.getCount(); i++) {
                int idx = om.getRequestIndex(i);
                std::cout << (i + 1) << ". " << station.getLibraryItem(idx)-
>getDescription() << "\n";
            }
        }
        break;
    }
    case 8: {
        bool isReq = false;
        int idx = station.playNext(currentDay, isReq);
        if (idx == -1) std::cout << "\n[AIR] -Silence- No active content.\n";
        else {
            std::cout << "\n>>> AIR BROADCASTING NOW <<<\n";
            if (isReq) std::cout << "[LISTENER ORDER] ";
            else std::cout << "[ROUTINE PLAN] ";
            std::cout << station.getLibraryItem(idx)->getDescription() << "\n";
        }
    }
}

```

```

        break;
    }
    case 9: {
        int d; std::cout << "Enter new day code (0-6): "; std::cin >> d;
        if (d >= 0 && d <= 6) { currentDay = d; std::cout << "Updated.\n"; }
        else std::cout << "Invalid code!\n";
        break;
    }
    default: std::cout << "Unknown command.\n";
}
std::cout << "\n";
}
}
};

// =====
// MAIN APPLICATION ENTRY POINT
// =====
int main() {
    // Execute automated tests first to prove system reliability
    TestingModule::runAutomatedTests();

    // Initialize core business entity
    RadioStation mainRadioStation;

    // Calling the helper functions from connected header files
    loadPopularSongs(mainRadioStation); // Automatically reads and fills songs
    loadCommercialProposals(mainRadioStation); // Automatically reads and fills
commercial
commercial

    // Pre-scheduling two items for Monday demo
    mainRadioStation.addTrackToSchedule(0, 5); // Mon: Song at index 5
    mainRadioStation.addTrackToSchedule(0, 52); // Mon: Commercial at index 52

    // Hand over control to user interaction module
    UserInterface ui;
    ui.showMainMenu(mainRadioStation);

    return 0;
}

```

Додаток Б. Лістинг коду програми файлу SongsData.h

```
#ifndef SONGS_DATA_H
#define SONGS_DATA_H

#include <string>

// Forward declaration of RadioStation class so the compiler knows it exists
class RadioStation;

// Helper function to seed 50 popular tracks into the radio library
void loadPopularSongs(RadioStation& station) {
    // Top hits from various genres and eras
    station.addSongToLibrary("Blinding Lights", "The Weeknd", 200);
    station.addSongToLibrary("Shape of You", "Ed Sheeran", 233);
    station.addSongToLibrary("Bohemian Rhapsody", "Queen", 355);
    station.addSongToLibrary("Stay", "The Kid LAROI & Justin Bieber", 141);
    station.addSongToLibrary("As It Was", "Harry Styles", 167);
    station.addSongToLibrary("Bad Guy", "Billie Eilish", 194);
    station.addSongToLibrary("Perfect", "Ed Sheeran", 263);
    station.addSongToLibrary("Believer", "Imagine Dragons", 204);
    station.addSongToLibrary("Levitating", "Dua Lipa", 203);
    station.addSongToLibrary("Flowers", "Miley Cyrus", 200);

    station.addSongToLibrary("Starboy", "The Weeknd", 230);
    station.addSongToLibrary("Someone You Loved", "Lewis Capaldi", 182);
    station.addSongToLibrary("Sweater Weather", "The Neighbourhood", 240);
    station.addSongToLibrary("Another One Bites the Dust", "Queen", 215);
    station.addSongToLibrary("Smells Like Teen Spirit", "Nirvana", 301);
    station.addSongToLibrary("Hotel California", "Eagles", 390);
    station.addSongToLibrary("Billie Jean", "Michael Jackson", 294);
    station.addSongToLibrary("In the End", "Linkin Park", 216);
    station.addSongToLibrary("Counting Stars", "OneRepublic", 257);
    station.addSongToLibrary("Wake Me Up", "Avicii", 247);

    station.addSongToLibrary("Riptide", "Vance Joy", 204);
    station.addSongToLibrary("Radioactive", "Imagine Dragons", 186);
    station.addSongToLibrary("Stressed Out", "Twenty One Pilots", 202);
    station.addSongToLibrary("Take Me to Church", "Hozier", 241);
    station.addSongToLibrary("Shallow", "Lady Gaga & Bradley Cooper", 215);
    station.addSongToLibrary("Love Yourself", "Justin Bieber", 233);
    station.addSongToLibrary("7 Rings", "Ariana Grande", 178);
    station.addSongToLibrary("Closer", "The Chainsmokers", 244);
    station.addSongToLibrary("Thinking Out Loud", "Ed Sheeran", 281);
    station.addSongToLibrary("Say You Won't Let Go", "James Arthur", 211);

    station.addSongToLibrary("Dynamite", "BTS", 199);
    station.addSongToLibrary("Cruel Summer", "Taylor Swift", 178);
    station.addSongToLibrary("Mockingbird", "Eminem", 251);
    station.addSongToLibrary("Without Me", "Eminem", 290);
    station.addSongToLibrary("Lose Yourself", "Eminem", 326);
    station.addSongToLibrary("Till I Collapse", "Eminem", 297);
    station.addSongToLibrary("Numb", "Linkin Park", 185);
    station.addSongToLibrary("Chop Suey!", "System of a Down", 210);
    station.addSongToLibrary("Calm Down", "Rema & Selena Gomez", 239);
    station.addSongToLibrary("Drivers License", "Olivia Rodrigo", 242);
```

```
station.addSongToLibrary("Good 4 U", "Olivia Rodrigo", 178);
station.addSongToLibrary("Heat Waves", "Glass Animals", 235);
station.addSongToLibrary("Cold Heart", "Elton John & Dua Lipa", 202);
station.addSongToLibrary("Industry Baby", "Lil Nas X & Jack Harlow", 212);
station.addSongToLibrary("Montero", "Lil Nas X", 137);
station.addSongToLibrary("Watermelon Sugar", "Harry Styles", 174);
station.addSongToLibrary("Circles", "Post Malone", 215);
station.addSongToLibrary("Sunflower", "Post Malone & Swae Lee", 158);
station.addSongToLibrary("Rockstar", "Post Malone", 218);
station.addSongToLibrary("One Dance", "Drake", 174);
}

#endif
```

Додаток В. Лістинг коду програми файлу **CommercialsData.h**

```
#ifndef COMMERCIALS_DATA_H
#define COMMERCIALS_DATA_H

#include <string>

// Forward declaration of RadioStation class
class RadioStation;

// Helper function to seed 10 global advertising blocks into the radio library
void loadCommercialProposals(RadioStation& station) {
    // List of business advertising campaigns
    station.addCommercialToLibrary("Summer Refresh Promo", "Coca-Cola", 30);
    station.addCommercialToLibrary("Just Do It Campaign", "Nike", 15);
    station.addCommercialToLibrary("iPhone New Release", "Apple", 45);
    station.addCommercialToLibrary("Galaxy Unpacked Event", "Samsung", 30);
    station.addCommercialToLibrary("I'm Lovin It Menu", "McDonalds", 20);
    station.addCommercialToLibrary("Max Taste No Sugar", "PepsiCo", 15);
    station.addCommercialToLibrary("Impossible Is Nothing", "Adidas", 30);
    station.addCommercialToLibrary("Ultimate Driving Machine", "BMW", 45);
    station.addCommercialToLibrary("PlayStation 5 Gaming", "Sony", 30);
    station.addCommercialToLibrary("Energy Boost Block", "Red Bull", 20);
}

#endif
```

Додаток Г. Результати роботи програми (Скріншоти верифікації системи)

Рисунок Г.1 - Вікно консолі при запуску програми: демонстрація успішного проходження автоматичних тестів TestingModule та завантаження 60 елементів з підключених файлів SongsData.h та CommercialsData.h.

```
[TEST MODULE] Starting automated architectural tests...
-> TEST 1 PASSED: Library counters valid.
-> TEST 2 PASSED: Commercial block request rejected successfully.
[TEST MODULE] Testing complete.
```

```
=====
      RADIO AIS INTERACTIVE UI
=====
Current Day: Monday
-----
1. Print Media Library
2. Add New Song
3. Add New Commercial Ad
4. Print Today's Schedule
5. Add Item to Day Schedule
6. Listener Mode: Order a Song
7. Print Active Requests Queue
8. Air Control: Play Next Track
9. Switch Current Day
0. Shut Down Station
-----
Select menu item:
```

Рисунок Г.2 - Демонстрація виведення повної медіатеки (пункт меню 1) зі зчитаними 50 піснями та 10 рекламними блоками.

```
-----  
Select menu item: 1  
  
--- MEDIA STATION LIBRARY ---  
[0] [Song] The Weeknd - Blinding Lights (200s)  
[1] [Song] Ed Sheeran - Shape of You (233s)  
[2] [Song] Queen - Bohemian Rhapsody (355s)  
[3] [Song] The Kid LAROI & Justin Bieber - Stay (141s)  
[4] [Song] Harry Styles - As It Was (167s)  
[5] [Song] Billie Eilish - Bad Guy (194s)  
[6] [Song] Ed Sheeran - Perfect (263s)  
[7] [Song] Imagine Dragons - Believer (204s)  
[8] [Song] Dua Lipa - Levitating (203s)  
[9] [Song] Miley Cyrus - Flowers (200s)  
[10] [Song] The Weeknd - Starboy (230s)  
[11] [Song] Lewis Capaldi - Someone You Loved (182s)  
[12] [Song] The Neighbourhood - Sweater Weather (240s)  
[13] [Song] Queen - Another One Bites the Dust (215s)  
[14] [Song] Nirvana - Smells Like Teen Spirit (301s)  
[15] [Song] Eagles - Hotel California (390s)  
[16] [Song] Michael Jackson - Billie Jean (294s)  
[17] [Song] Linkin Park - In the End (216s)  
[18] [Song] OneRepublic - Counting Stars (257s)  
[19] [Song] Avicii - Wake Me Up (247s)  
[20] [Song] Vance Joy - Riptide (204s)  
[21] [Song] Imagine Dragons - Radioactive (186s)  
[22] [Song] Twenty One Pilots - Stressed Out (202s)  
[23] [Song] Hozier - Take Me to Church (241s)  
[24] [Song] Lady Gaga & Bradley Cooper - Shallow (215s)  
[25] [Song] Justin Bieber - Love Yourself (233s)  
[26] [Song] Ariana Grande - 7 Rings (178s)  
[27] [Song] The Chainsmokers - Closer (244s)  
[28] [Song] Ed Sheeran - Thinking Out Loud (281s)  
[29] [Song] James Arthur - Say You Won't Let Go (211s)  
[30] [Song] BTS - Dynamite (199s)  
[31] [Song] Taylor Swift - Cruel Summer (178s)  
[32] [Song] Eminem - Mockingbird (251s)  
[33] [Song] Eminem - Without Me (290s)  
[34] [Song] Eminem - Lose Yourself (326s)  
[35] [Song] Eminem - Till I Collapse (297s)
```

[36] [Song] Linkin Park - Numb (185s)
[37] [Song] System of a Down - Chop Suey! (210s)
[38] [Song] Rema & Selena Gomez - Calm Down (239s)
[39] [Song] Olivia Rodrigo - Drivers License (242s)
[40] [Song] Olivia Rodrigo - Good 4 U (178s)
[41] [Song] Glass Animals - Heat Waves (235s)
[42] [Song] Elton John & Dua Lipa - Cold Heart (202s)
[43] [Song] Lil Nas X & Jack Harlow - Industry Baby (212s)
[44] [Song] Lil Nas X - Montero (137s)
[45] [Song] Harry Styles - Watermelon Sugar (174s)
[46] [Song] Post Malone - Circles (215s)
[47] [Song] Post Malone & Swae Lee - Sunflower (158s)
[48] [Song] Post Malone - Rockstar (218s)
[49] [Song] Drake - One Dance (174s)
[50] [Commercial] Summer Refresh Promo (Sponsor: Coca-Cola, 30s)
[51] [Commercial] Just Do It Campaign (Sponsor: Nike, 15s)
[52] [Commercial] iPhone New Release (Sponsor: Apple, 45s)
[53] [Commercial] Galaxy Unpacked Event (Sponsor: Samsung, 30s)
[54] [Commercial] I'm Lovin It Menu (Sponsor: McDonalds, 20s)
[55] [Commercial] Max Taste No Sugar (Sponsor: PepsiCo, 15s)
[56] [Commercial] Impossible Is Nothing (Sponsor: Adidas, 30s)
[57] [Commercial] Ultimate Driving Machine (Sponsor: BMW, 45s)
[58] [Commercial] PlayStation 5 Gaming (Sponsor: Sony, 30s)
[59] [Commercial] Energy Boost Block (Sponsor: Red Bull, 20s)

=====

RADIO AIS INTERACTIVE UI

=====

Current Day: Monday

1. Print Media Library
 2. Add New Song
 3. Add New Commercial Ad
 4. Print Today's Schedule
 5. Add Item to Day Schedule
 6. Listener Mode: Order a Song
 7. Print Active Requests Queue
 8. Air Control: Play Next Track
 9. Switch Current Day
 0. Shut Down Station
-

Рисунок Г.3 - Демонстрація роботи валідатора замовлень (пункт меню 6):
успішна реєстрація музичного треку та відхилення спроби замовити рекламу.

```
=====
RADIO AIS INTERACTIVE UI
=====
Current Day: Monday
-----
1. Print Media Library
2. Add New Song
3. Add New Commercial Ad
4. Print Today's Schedule
5. Add Item to Day Schedule
6. Listener Mode: Order a Song
7. Print Active Requests Queue
8. Air Control: Play Next Track
9. Switch Current Day
0. Shut Down Station
-----
Select menu item: 6
Enter Song index to request (0-59): 48
Success! Registered.
```

```
=====
RADIO AIS INTERACTIVE UI
=====
Current Day: Monday
-----
1. Print Media Library
2. Add New Song
3. Add New Commercial Ad
4. Print Today's Schedule
5. Add Item to Day Schedule
6. Listener Mode: Order a Song
7. Print Active Requests Queue
8. Air Control: Play Next Track
9. Switch Current Day
0. Shut Down Station
-----
Select menu item: 6
Enter Song index to request (0-59): 58
Rejected: Invalid index OR item is an Ad!
```

Рисунок Г.4 - Демонстрація симуляції ефіру (пункт меню 8): вихід першочергового замовлення від слухача за алгоритмом FIFO, а за його відсутності перехід до планової сітки мовлення радіостанції.

```
=====
Current Day: Monday
-----
1. Print Media Library
2. Add New Song
3. Add New Commercial Ad
4. Print Today's Schedule
5. Add Item to Day Schedule
6. Listener Mode: Order a Song
7. Print Active Requests Queue
8. Air Control: Play Next Track
9. Switch Current Day
0. Shut Down Station
-----
Select menu item: 8

>>> AIR BROADCASTING NOW <<<
[LISTENER ORDER] [Song] Post Malone - Rockstar (218s)

=====
      RADIO AIS INTERACTIVE UI
=====
Current Day: Monday
-----
1. Print Media Library
2. Add New Song
3. Add New Commercial Ad
4. Print Today's Schedule
5. Add Item to Day Schedule
6. Listener Mode: Order a Song
7. Print Active Requests Queue
8. Air Control: Play Next Track
9. Switch Current Day
0. Shut Down Station
-----
Select menu item: 8

>>> AIR BROADCASTING NOW <<<
[ROUTINE PLAN] [Song] Billie Eilish - Bad Guy (194s)
```